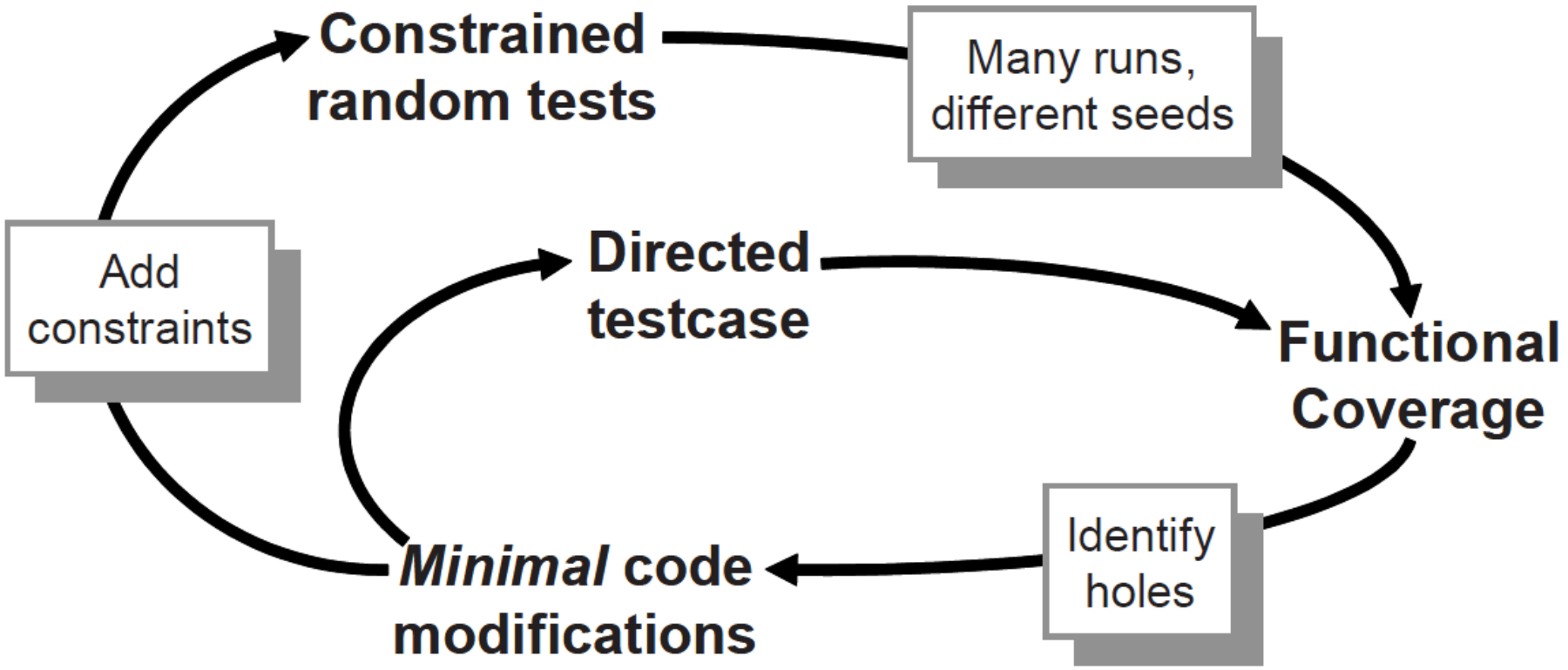
Functional Coverage

- As designs become more complex, the only effective way to verify them thoroughly is with constrained-random testing (CRT).
- This approach elevates you above the tedium of writing individual directed tests, one for each feature in the design.
- However, if your testbench is taking a random walk through the space of all design states, how do you know if you have reached your destination?
- Whether you are using random or directed stimulus, you can gauge progress using coverage.

- Functional coverage is a measure of which design features have been exercised by the tests.
- Start with the design specification and create a verification plan with a detailed list of what to test and how.

- For example, if your design connects to a bus, your tests need to exercise all the possible interactions between the design and bus, including relevant design states, delays, and error modes.
- The verification plan is a map to show you where to go.

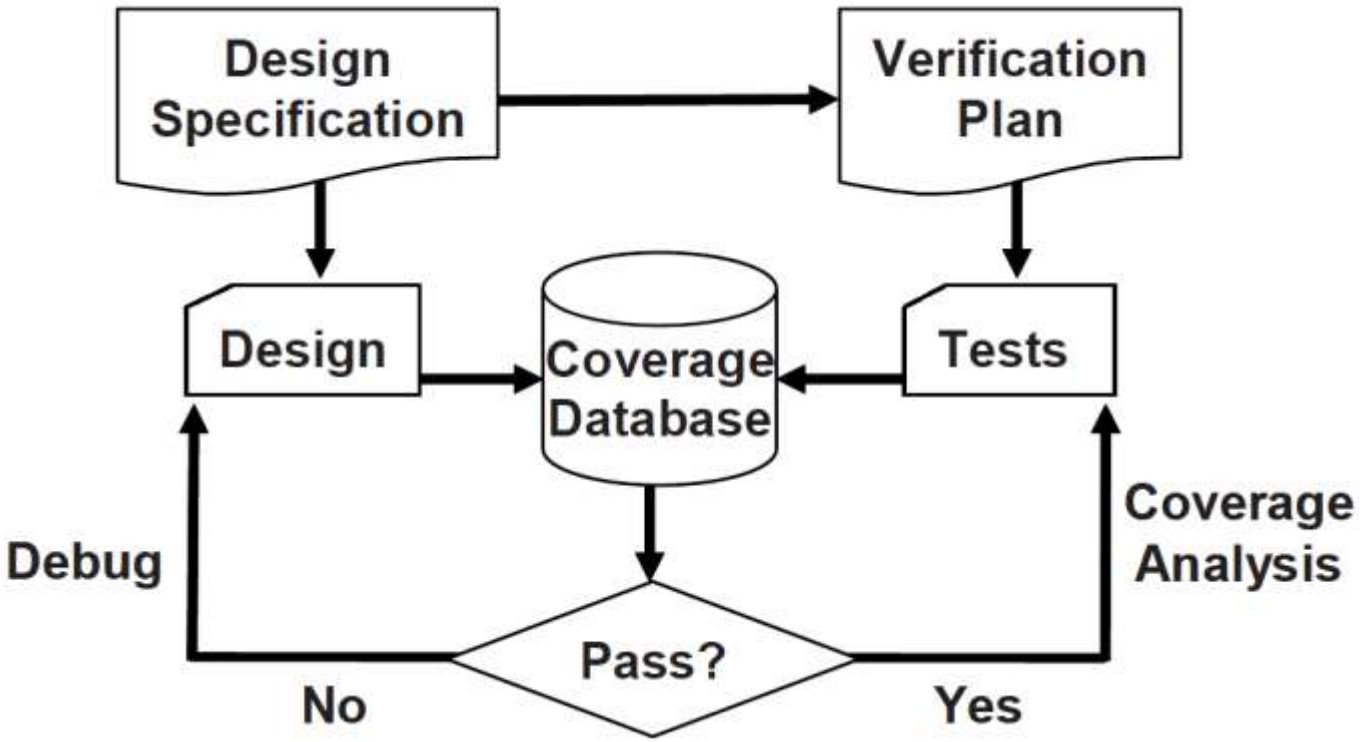
- Use a feedback loop to analyze the coverage results and decide on which actions to take in order to converge on 100% coverage.
- Your first choice is to run existing tests with more seeds; the second is to build new constraints.
- Resort to creating directed tests only if absolutely necessary.



- With CRT, you are freed from hand crafting every line of input stimulus, but now you need to write code that tracks the effectiveness of the test with respect to the verification plan.
- You are still more productive, as you are working at a higher level of abstraction.
- You have moved from tweaking individual bits to describing the interesting design states.
- Reaching for 100% functional coverage forces you to think more about what you want to observe and how you can direct the design into those states.

Gathering Coverage Data

- You can run the same random testbench over and over, simply by changing the random seed, to generate new stimulus.
- Each individual simulation generates a database of functional coverage information, the trail of footprints from the random walk.
- You can then merge all this information together to measure your overall progress using functional coverage



- You then analyze the coverage data to decide how to modify your tests.
- If the coverage levels are steadily growing, you may just need to run existing tests with new random seeds, or even just run longer tests.
- If the coverage growth has started to slow, you can add additional constraints to generate more “interesting” stimuli.
- When you reach a plateau, some parts of the design are not being exercised, and so you need to create more tests.
- Lastly, when your functional coverage values near 100%, check the bug rate.
- If bugs are still being found, you may not be measuring true coverage for some areas of your design.
- Don’t be in too big of a rush to reach 100% coverage, which just shows that you looked for bugs in all the usual places.

- While you are trying to verify your design, take many random walks through the stimulus space; this can create many unanticipated combinations

Each simulation vendor has its own format for storing coverage data and as well as its own analysis tools. You need to perform the following actions with those tools.

- Run a test with multiple seeds. For a given set of constraints (and coverage groups), compile the testbench and design into a single executeable.
- Now you need to run this constraint set over and over with different random seeds.

  - Check for pass/fail. Functional coverage information is only valid for a successful simulation.
  - When a simulation fails because there is a design bug, the coverage information must be discarded.
  - The coverage data measures how many items in the verification plan are complete, and this plan is based on the design specification.
  - If the design does not match the specification, the coverage values are useless.

  - Analyze coverage across multiple runs.
  - You need to measure how successful each constraint set is, over time.
  - If you are not yet getting 100% coverage for the areas that are targeted by the constraints, but the amount is still growing,run more seeds.
  - If the coverage level has plateaued, with no recent progress, it is time to modify the constraints.

Code Coverage

- The easiest way to measure verification progress is with code coverage.
- Here you are measuring how many lines of code have been executed (line coverage), which paths through the code and expressions have been executed (path coverage), which singlebit variables have had the values 0 or 1 (toggle coverage), and which states and transitions in a state machine have been visited (FSM coverage).
- You don’t have to write any extra HDL code.
- The tool instruments your design automatically by analyzing the source code and adding hidden code to gather statistics.

- Code coverage measures how thoroughly your tests exercised the “implementation” of the design specification, and not the verification plan.
- Just because your tests have reached 100% code coverage, your job is not done. What if you made a mistake that your test didn’t catch?
- Worse yet, what if your implementation is missing a feature?

```
module dff(output logic q, q_l,
           input  logic clk, d, reset_l);

  always @(posedge clk or negedge reset_l) begin
    q <= d;
    q_l <= !d;
  end
endmodule
```

The reset logic was accidently left out. A code coverage tool would report that every line had been exercised, yet the model was not implemented correctly.

Functional Coverage

- The goal of verification is to ensure that a design behaves correctly in its real environment, be that an MP3 player, network router, or cell phone.
- The design specification details how the device should operate, whereas the verification plan lists how that functionality is to be stimulated, verified, and measured.
- When you gather measurements on what functions were covered, you are performing “design” coverage.

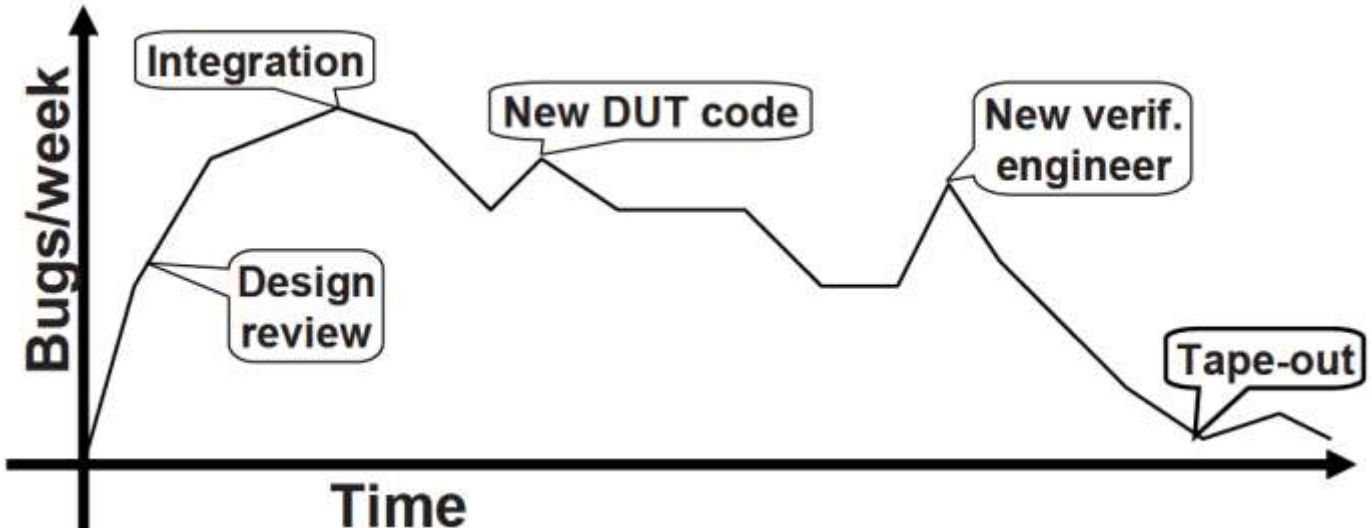
- Forexample, the verification plan for a D-flip flop would mention not only its data storage but also how it resets to a known state.
- Until your test checks both these design features, you will not have 100% functional coverage.

- Functional coverage is tied to the design intent and is sometimes called “specification coverage,” while code coverage measures the design implementation.

Bug Rate

- An indirect way to measure coverage is to look at the rate at which fresh bugs are found.
- You should keep track of how many bugs you found each week, over the life of a project.
- At the start, you may find many bugs through inspection as you create the testbench.
- As you read the design spec, you may find inconsistencies, which hopefully are fixed before the RTL is written.
- Once the testbench is up and running, a torrent of bugs is found as you check each module in the system.
- The bug rate drops, hopefully to zero, as the design nears tape-out.
- However, you are not yet done. Everytime the rate sags, it is time to find different ways to create corner cases.

- The bug rate can vary per week based on many factors such as project phases, recentdesign changes, blocks being integrated, personnel changes, and even vacation schedules.



Functional Coverage Strategies

- Before you write the first line of test code, you need to anticipate what are the key design features, corner cases, and possible failure modes.
- This is how you write your verification plan.
- Don’t think in terms of data values only; instead, think about what information is encoded in the design.
- The plan should spell out the significant design

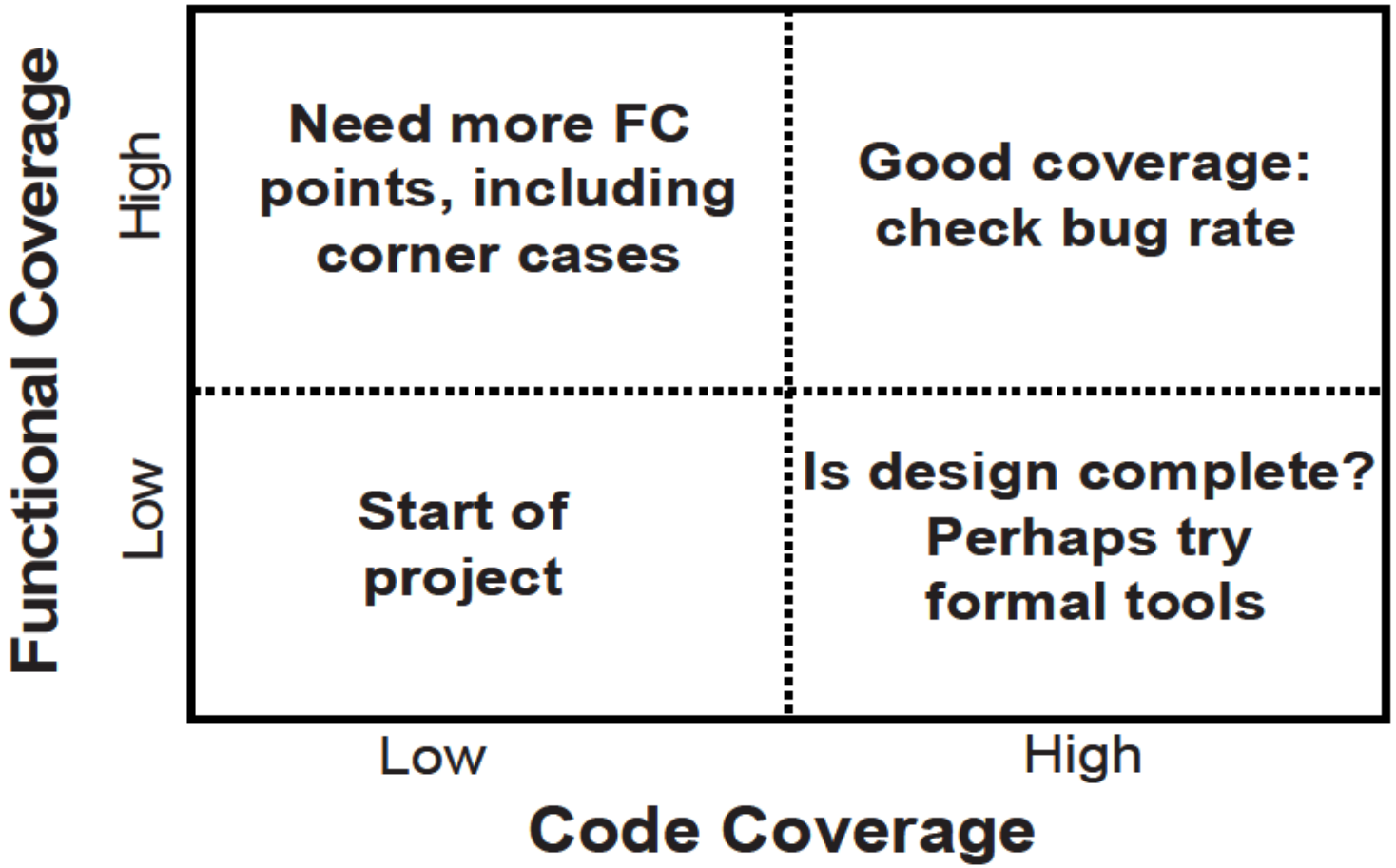
Gather Information, Not Data

Only Measure What You Are Going to Use

- Gathering functional coverage data can be expensive, and so only measure what you will analyze and use to improve your tests.
- Your simulations may run slower as the simulator monitors signals for functional coverage.

Measuring Completeness

- your manager constantly asks you, “Are we there yet?”
- How can you tell if you have fully tested a design?
- You need to look at all coverage measurements and consider the bug rate to see if you have reached your destination.



- At the start of a project, both code and functional coverage are low.
- As you develop tests, run them over and over with different random seeds until you no longer see increasing values of functional coverage.
- Create additional constraints and tests to explore new areas

- What if the functional coverage is high but the code coverage is low?
- Your tests are not exercising the full design, perhaps from an inadequate verification plan.
- It may be time to go back to the hardware specifications and update your verification plan.
- Then you need to add more functional coverage points to locate untested functionality

- A more difficult situation is high code coverage but low functional coverage.
- Even though your testbench is giving the design a good workout, you are unable to put it in all the interesting states.
- First, see if the design implements all the specified functionality.
- If the functionality is there, but your tests can’t reach it, you might need a formal verification tool that can extract the design’s states and create appropriate stimulus.

- The goal is both high code and functional coverage.
- However, don’t plan your vacation yet.
- What is the trend of the bug rate?
- Are significant bugs still popping up?
- Worse yet, are they being found deliberately, or did your testbench happen to stumble across a particular combination of states that no one had anticipated?
- On the other hand, a low bug rate may mean that your existing strategies have run out of steam, and you should look into different approaches.
- Try different approaches such as new combinations of design blocks and error generators.

Simple Functional Coverage Example

- To measure functional coverage, you begin with the verification plan and write an executable version of it for simulation.
- In your SystemVerilog testbench, sample the values of variables and expressions.
- These sample locations are known as cover points.
- Multiple cover points that are sampled at the same time (such as when a transaction completes) are placed together in a cover group

```
program automatic test(busifc.TB ifc);

  class Transaction;
    rand bit [31:0] data;
    rand bit [ 2:0] port;          // Eight port numbers
  endclass

  covergroup CovPort;
    coverpoint tr.port;            // Measure coverage
  endgroup

  initial begin
    Transaction tr;
    CovPort ck;
    ck = new();                     // Instantiate group
    tr = new();
    repeat (32) begin               // Run a few cycles
      assert(tr.randomize);         // Create a transaction
      ifc.cb.port <= tr.port;       //   and transmit
      ifc.cb.data <= tr.data;       //   onto interface
      ck.sample();                  // Gather coverage
      @ifc.cb;                      // Wait a cycle
    end
  end
endprogram
```

- creates a random transaction and drives it out to an interface.
- The testbench samples the value of the port field using the CovPort cover group.
- Eight possible values, 32 random transactions – did your testbench generate them all?

```
Coverpoint Coverage report
CoverageGroup: CovPort
    Coverpoint: tr.port
Summary
    Coverage: 87.50
    Goal: 100
    Number of Expected auto-bins: 8
    Number of User Defined Bins: 0
    Number of Automatically Generated Bins: 7
    Number of User Defined Transitions: 0

    Automatically Generated Bins
```

```
 Automatically Generated Bins

 Bin              # hits     at least
 ================================
 auto[1]          7          1
 auto[2]          7          1
 auto[3]          1          1
 auto[4]          5          1
 auto[5]          4          1
 auto[6]          2          1
 auto[7]          6          1
 ================================
```

- As you can see, the testbench generated the values 1, 2, 3, 4, 5, 6, and 7, but never
- generated a port of 0.
- The at least column specifies how many hits are needed before a bin is considered covered

- To improve your functional coverage, the easiest strategy is to just run more simulation cycles, or to try new random seeds.
- Look at the coverage report for items with two or more hits.
- Chances are that you just need to make the simulation run longer or to try new seed values.
- If a cover point had zero or one hit, you probably have to try a new strategy, as the testbench is not creating the proper stimulus.

For this example, the very next random transaction (#33) has a `port` value of 0, giving 100% coverage.

```
Coverpoint Coverage report
CoverageGroup: CovPort
    Coverpoint: tr.port
Summary
    Coverage: 100
    Goal: 100
    Number of Expected auto-bins: 8
    Number of User Defined Bins: 0
    Number of Automatically Generated Bins: 8
    Number of User Defined Transitions: 0
```

```
Automatically Generated Bins

Bin             # hits    at least
================================
auto[0]         1         1
auto[1]         7         1
auto[2]         7         1
auto[3]         1         1
auto[4]         5         1
auto[5]         4         1
auto[6]         2         1
auto[7]         6         1
================================
```

Anatomy of a Cover Group

- A cover group is similar to a class – you define it once and then instantiate it one or more times.
- It contains cover points, options, formal arguments, and an optional trigger.
- A cover group encompasses one or more data points, all of which are sampled at the same time.
- You should create very clear cover group names that explicitly indicate what you are measuring and, if possible, reference to the verification plan.
- A cover group can be defined in a class or at the program or module level.
- It can sample any visible variable such as program/module variables, signals from an interface, or any signal in the design (using a hierarchical reference).
- A cover group inside a class can sample variables in that class, as well as data values from embedded classes.

- Don’t define the cover group in a data class, such as a transaction, as doing so can cause additional overhead when gathering coverage data.
- Imagine you are trying to track how many Coldrinks were consumed by patrons in a cafe.
- Would you try to follow every bottle as it flowed from the loading dock, over the cafe, and into each person?
- No, instead you could just have each patron check off the type and number of coldrinks consumed,

Defining the coverage model

- The coverage model is defined using Covergroup construct.
- The covergroup construct is a user-defined type.
- The type definition is written once, and multiple instances of that type can be created in different contexts.
- Similar to a class, once defined, a covergroup instance can be created via the new()operator.
- A covergroup can be defined in a module, program, interface, or class.

Each covergroup specification can include,

- A clocking event that synchronizes the sampling of coverage points
- A set of coverage points
- Cross coverage between coverage points
- Optional formal arguments
- Coverage options

Functional Coverage example

Example-1

```
covergroup cov_grp @(posedge clk);
  cov_p1: coverpoint a;
endgroup

cov_grp cov_inst = new();
```

In the example-1 clocking, event specifies the event at which coverage points are sampled.

Example-2

```
covergroup cov_grp;
  cov_p1: coverpoint a;
endgroup

cov_grp cov_inst = new();
@(abc) cov_inst.sample();
```

In the example-2 coverage, sampling is triggered by calling a built-in sample() method.

Defining coverage points

- A covergroup can contain one or more coverage points.
- A coverage point can be an integral variable or an integral expression.
- Each coverage point is associated with “bin”.
- On each sample clock simulator will increment the associated bin value.

The bins will automatically be created or can be explicitly defined.



Automatic Bins or Implicit Bins

- An automatically single bin will be created for each value of the coverpoint variable range.
- These are called automatic, or implicit, bins.
- For an “n” bit integral coverpoint variable, a 2^n number of automatic bins will get created.

```
module cov;
  logic    clk;
  logic [7:0] addr;
  logic    wr_rd;

  covergroup cg @(posedge clk);
    c1: coverpoint addr;
    c2: coverpoint wr_rd;
  endgroup : cg
  cg cover_inst = new();
  ...
endmodule
```

for addr: c1.auto[0] c1.auto[1] c1.auto[2] … c1.auto[255]
for wr_rd: c2.auto[0]

Explicit bins

- “bins” keyword is used to declare the bins explicitly to a variable.
- A separate bin is created for each value in the given range of variable or a single/multiple bins for the rage of values.
- Bins are explicitly declared within curly braces { } along with the bins keyword followed by bin name and variable value/range, immediately after the coverpoint identifier.

```
module cov;
  logic     clk;
  logic [7:0] addr;
  logic     wr_rd;

  covergroup cg @(posedge clk);
    c1: coverpoint addr { bins b1 = {0,2,7};
                          bins b2[3] = {11:20};
                          bins b3    = {[30:40],[50:60],77};
                          bins b4[] = {[79:99],[110:130],140};
                          bins b5[] = {160,170,180};
                          bins b6     = {200:$};
                          bins b7 = default;}
    c2: coverpoint wr_rd {bins wrrd};
  endgroup : cg

  cg cover_inst = new();
  ...
endmodule
```

```
bins b1     = {0,2,7 };                  //bin “b1” increments for addr = 0,2 or 7
bins b2[3] = {11:20};                   //creates three bins b2[0],b2[1] and b2[3].and The 11
possible values are
                                         //distributed as follows: (11,12,13),(14,15,16) and
(17,18,19,20) respectively.
bins b3     = {[30:40],[50:60],77};     //bin “b3” increments for addr = 30-40 or 50-60 or 77
bins b4[] = {[79:99],[110:130],140};//creates three bins b4[0],b4[1] and b4[3] with values
79-99,50-60 and 77 respectively
bins b5[] = {160,170,180};              //creates three bins b5[0],b5[1] and b5[3] with values
160,170 and 180 respectively
bins b6     = {200:$};                  //bin “b6” increments for addr = 200 to max value i.e,
255.
default bin;                            // catches the values of the coverage point that do not
lie within any of the defined bins.
```

How is functional coverage done in SystemVerilog ?

The idea is to sample interesting variables in the testbench and analyze if they have reached certain set of values.

```
module test;
  bit [3:0] mode;
  bit [1:0] key;

  // Other testbench code
endmodule
```

- mode can take 16 values, while key can take 4 values.
- So, if there's something to monitor these two variables in a simulation and report what values of mode and key have been exercised, you'll know if the test covered a particular feature or not.
- The good part is that there are options in a simulator to dump out such coverage details into a file so that it can be reviewed after the simulation has finished.
- Moreover, you can merge all such coverage files from different tests into a single database and review them as a whole.
- If test A covered feature X and test B covered feature Y, the merged database will show that you have covered both X and Y.

How to write covergroups ?

```
class myTrns;
  rand bit [3:0]   mode;
  rand bit [1:0]   key;

   function display ();
      $display ("[%0tns] mode = 0x%0h, key = 0x%0h", $time, mode, key);
   endfunction

  covergroup CovGrp;
    coverpoint mode {
      bins featureA   = {0};
      bins featureB   = {[1:3]};
      bins common []   = {4:$};
      bins reserve  = default;
    }
    coverpoint key;
  endgroup
endclass
```

- Variables are mentioned as a coverpoint.
- Coverpoints are put together in a covergroup block.
- Multiple covergroups can be created to sample the same variables with different set of bins
- bins are said to be "hit/covered" when the variable reaches the corresponding values. So, the bin featureB is hit when mode takes either 1,2 or 3.
- bin reserve is a single bin for all values that do not fall under the other bins.
- **common** will have 4 separate bins, one for each value.

Why are coverage metrics missing in simulations ?

You have to enable the tool vendor specific command-line switch to dump coverage details.
Then open a coverage viewer tool like Cadence ICCR/IMC and open the coverage dump file.

How to specify when to sample ?

There are two ways to trigger coverage collection in a covergroup.

Use sample() method of a particular covergroup to sample coverpoints within that group

```
class myCov;
  covergroup CovGrp;
     ...
  endgroup

  function new ();
    CovGrp = new;              // Create an instance of the covergroup
  endfunction
endclass

module tb_top;
  myCov myCov0 = new ();       // Create an instance of the class

  initial begin
    myCov0.CovGrp.sample ();
  end
endmodule
```

Mention the event at which the covergroup should be sampled

```
covergroup CovGrp @ (posedge clk);    // Sample coverpoints at posedg
covergroup CovGrp @ (eventA);         // eventA can be triggered with
```

What are the ways for conditional coverage ?

Yes, you have two ways to conditionally enable coverage.

- Use `iff` construct

```
covergroup CovGrp;
  coverpoint mode iff (!_if.reset) {
      // bins for mode
  }
endgroup
```

- Use `start` and `stop` functions

```
CovGrp cg = new;

initial begin
  #1 _if.reset = 0;
  cg.stop ();
  #10 _if.reset = 1;
  cg.start();
end
```

SystemVerilog Covergroup and Coverpoint

- SystemVerilog covergroup is a user-defined type that encapsulates the specification of a coverage model. They can be defined once and instantiated muliple times at different places via the new function.
- covergroup can be defined in either a package, module, program, interface, or class and usually encapsulates the following information:

  - A set of coverage points
  - Cross coverage between coverage points
  - An event that defines when the covergroup is sampled
  - Other options to configure coverage object

```
module tb;

  // Declare some variables that can be "sampled" in the covergroup
  bit [1:0] mode;
  bit [2:0] cfg;

  // Declare a clock to act as an event that can be used to sample
  // coverage points within the covergroup
  bit clk;
  always #20 clk = ~clk;

  // "cg" is a covergroup that is sampled at every posedge clk
  covergroup cg @ (posedge clk);
    coverpoint mode;
  endgroup

  // Create an instance of the covergroup
  cg  cg_inst;
```

```
  initial begin
    // Instantiate the covergroup object similar to a class object
    cg_inst= new();

    // Stimulus : Simply assign random values to the coverage variables
    // so that different values can be sampled by the covergroup object
    for (int i = 0; i < 5; i++) begin
      @(negedge clk);
      mode = $random;
      cfg  = $random;
      $display ("[%0t] mode=0x%0h cfg=0x%0h", $time, mode, cfg);
    end
  end

  // At the end of 500ns, terminat
  initial begin
    #500 $display ("Coverage = %0.
    $finish;
  end
endmodule
```

```
ncsim> run
[40] mode=0x0 cfg=0x1
[80] mode=0x1 cfg=0x3
[120] mode=0x1 cfg=0x5
[160] mode=0x1 cfg=0x2
[200] mode=0x1 cfg=0x5
Coverage = 50.00 %
Simulation complete via $finish(1) at time 500 NS + 0
```

```
coverpoint mode {
    // Manually create a separate bin for each value
    bins zero = {0};
    bins one  = {1};

    // Allow SystemVerilog to automatically create separate bins for each value
    // Values from 0 to maximum possible value is split into separate bins
    bins range[] = {[0:$]};

    // Create automatic bins for both the given ranges
    bins c[] = { [2:3], [5:7]};

    // Use fixed number of automatic bins. Entire range is broken up into 4 bins
    bins range[4] = {[0:$]};

    // If the number of bins cannot be equally divided for the given range, then
    // the last bin will include remaining items; Here there are 13 values to be
    // distributed into 4 bins which yields:
    // [1,2,3] [4,5,6] [7,8,9] [10, 1, 3, 6]
    bins range[4] = {[1:10], 1, 3, 6};

    // A single bin to store all other values that don't belong to any other bin
    bins others = default;
  }
```

```
    module tb;
      bit [2:0] mode;

      // This covergroup does not get sample automatically because
      // the sample event is missing in declaration
      covergroup cg;
        coverpoint mode {
          bins one = {1};
          bins five = {5};
        }
      endgroup
// Stimulus : Simply randomize mode to have different values and
// manually sample each time
initial begin
  cg cg_inst = new();
  for (int i = 0; i < 5; i++) begin
  #10 mode = $random;
    $display ("[%0t] mode = 0x%0h", $time, mode);
    cg_inst.sample();
  end
  $display ("Coverage = %0.2f %%". cg_inst.get_inst_coverage());
end
                    endmodule
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 100.00 %
ncsim: *W,RNQUIE: Simulation is com
```

```
covergroup cg;
   coverpoint mode {

    // Declares a separate bin for each values -> Here there will 8 bins
    bins range[] = {[0:$]};
  }
 endgroup
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 50.00 %
ncsim: *W,RNQUIE: Simulation is complete.
```

4 out of the total possible 8 values have been sampled and hence coverage is 50%.

Fixed Number of automatic bins

```
covergroup cg;
  coverpoint mode {

    // Declares 4 bins for the total range of 8 values
    // So bin0->[0:1] bin1->[2:3] bin2->[4:5] bin3->[6:7]
    bins range[4] = {[0:$]};
  }
endgroup
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 75.00 %
ncsim: *W,RNQUIE: Simulation is complete.
```

`mode` never got a value of 6 or 7 and hence bin3 does not get hit. But all the other bins are hit and hence coverage is 75%.

Split fixed number of bins between a given range

```
covergroup cg;
  coverpoint mode {

    // Defines 3 bins
    // Two bins for values from 1:4, and one bin for value 7
    // bin1->[1,2] bin2->[3,4], bin3->7
    bins range[3] = {[1:4], 7};
  }
endgroup
```

```
ncsim> run
[10] mode = 0x4
[20] mode = 0x1
[30] mode = 0x1
[40] mode = 0x3
[50] mode = 0x5
Coverage = 66.67 %
ncsim: *W,RNQUIE: Simulation is complete.
```

Only 2/3 of the bins were hit and hence coverage is at 66.67%. To be specific, bin1 and bin2 were hit because `mode` was sampled to have 1 and [3,4] respectively.

COVER POINTS

- A covergroup can contain one or more coverage points.
- A coverage point can be an integral variable or an integral expression.
- A coverage point creates a hierarchical scope, and can be optionally labeled.
- If the label is specified then it designates the name of the coverage point.

```
program main;
bit [0:2] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y;
endgroup

cg cg_inst = new();

initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

- we are sampling the cover point "y".
- The cover point is named "cover_point_y" .
- In the Coverage report you will see this name.
- A cover group "cg" is defined and its instance "cg_inst" is created.
- The value of "y" is sampled when cg_inst.sample() method is called.
- Total possible values for Y are 0,1,2,3,4,5,6,7.
- The variable "y" is assigned only values 3,5,6.
- The coverage engine should report that only 3 values are covered and there are 8 possible values.

```
VARIABLE : cover_point_y
Expected : 8
Covered : 3
Percent: 37.50.
```

COVERPOINT EXPRESSION

- A coverage point can be an integral variable or an integral Expression.
- SystemVerilog allows specifying the cover points in various ways.

1)Using XMR

Example:

```
Cover_xmr : coverpoint top.DUT.Submodule.bus_address;
```

2)Part select

Example:

```
Cover_part: coverpoint bus_address[31:2];
```

3)Expression

Example:

```
Cocver_exp: coverpoint (a*b);
```

4)Function return value

Example:

```
Cover_fun: coverpoint funcation_call();
```

5)Ref variable

Example:

```
covergroup (ref int r_v) cg;
cover_ref: coverpoint r_v;
endgroup
```

Coverage Filter

- The expression within the iff construct specifies an optional condition that disables coverage for that cover point.
- If the guard expression evaluates to false at a sampling point, the coverage point is ignored.

```
covergroup cg;
coverpoint cp_varib iff(!reset); // filter condition
endgroup
```

- cover point varible "cp_varib" is covered only if the value reset is low.

GENERIC COVERAGE GROUPS

- Generic coverage groups can be written by passing their traits as arguments to the coverage constructor.
- This allows creating a reusable coverage group which can be used in multiple places.

  - To cover a array of specified index range.

```
covergroup cg(ref int array, int low, int high ) @(clk);
coverpoint// sample variable passed by reference
{
bins s = { [low : high] };
}
endgroup

int A, B;
rgc1 = new( A, 0, 50 );// cover A in range 0 to 50
rgc2 = new( B, 120, 600 );// cover B in range 120 to 600
```

- The example above defines a coverage group, gc, in which the signal to be sampled as well as the extent of the coverage bins are specified as arguments.
- Later, two instances of the coverage group are created each instance samples a different signal and covers a different range of values.

Implicit Bins

- While define cover point, if you do not specify any bins, then Implicit bins are created.
- The number of bins creating can be controlled by auto_bin_max parameter.

```
program main;
bit [0:2] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y
{ option.auto_bin_max = 4 ; }
endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Bin[0] for 0 and 1
Bin[1] for 2 and 3
Bin[2] for 4 and 5
Bin[3] for 6 and 7
```

```
Coverage report:
--------------------
VARIABLE : cover_point_y
Expected : 4
Covered : 3
Percent: 75.00

Uncovered bins
-----------------
auto[0:1]

Covered bins
-----------------
auto[2:3]
auto[4:5]
auto[6:7]
```

- For Enum data type, the numbers of bins are equal to the number of elements of enum data type.
- The bin identifiers are the enum member name.

```
typedef enum { A,B,C,D } alpha;
program main;
alpha y;
alpha values[$]= '{A,B,C};

covergroup cg;
cover_point_y : coverpoint y;
endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

- In The above example, the variable "y" is enum data type and it can have 4 enum members A,B,C and D.
- Variable Y is assigned only 3 Enum members A,B and C.

```
Coverage report:
--------------------
VARIABLE : cover_point_y
Expected : 4
Covered : 3
Percent: 75.00

Uncovered bins
--------------------
auto_D

Covered bins
--------------------
auto_C
auto_B
auto_A
```

EXPLICIT BIN CREATION

- Explicit bin creation is recommended method.
- Not all values are interesting or relevant in a cover point, so when the user knows the exact values he is going to cover, he can use explicit bins.
- You can also name the bins.

```
program main;
bit [0:2] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y {
bins a = {0,1};
bins b = {2,3};
bins c = {4,5};
bins d = {6,7};
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Coverage report:
------------------
VARIABLE : cover_point_y
Expected : 4
Covered : 3
Percent: 75.00

Uncovered bins
--------------------
a

Covered bins
--------------------
b
c
d
```

Array Of Bins

```
program main;
bit [0:2] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y {
bins a[] = {[0:7]};
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

In the above example, bin a is array of 8 bins and each bin associates to one number between 0 to 7.

```
Coverage report:
------------------
VARIABLE : cover_point_y
Expected : 8
Covered : 3
Percent: 37.50

Uncovered bins
------------------
a_0
a_1
a_2
a_4
a_7

Covered bins
------------------
a_3
a_5
a_6
```

To create a fixed number of bins for a set of values, a number can be specified inside the square brackets.

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y {
bins a[4] = {[0:7]};
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

In the above example, variable y is 4 bit width vector.
Total possible values for this vector are 16.
But in the cover point bins, we have giving the interested range as 0 to 7.
So the coverage report is calculated over the range 0 to 7 only. In this example,
we have shown the number bins to be fixed to size 4.

```
Coverage report:
------------------
VARIABLE : cover_point_y
Expected : 4
Covered : 3
Percent: 75.00

Uncovered bins
------------------
a[0:1]

Covered bins
------------------
a[2:3]
a[4:5]
a[6:7]
```

Default Bin

- The default specification defines a bin that is associated with none of the defined value bins.
- The default bin catches the values of the coverage point that do not lie within any of the defined bins.
- However, the coverage calculation for a coverage point shall not take into account the coverage captured by the default bin.

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y {
bins a[2] = {[0:4]};
bins d = default;
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

In the above example, we have specified only 2 bins to cover values from 0 to 4.
Rest of values are covered in default bin ~Sd~T which is not using in calculating the coverage percentage.

```
Coverage report:
------------------
VARIABLE : cover_point_y
Expected : 2
Covered : 1
Percent: 50.00

Uncovered bins
----------------
a[0:1]

Covered bins
--------------
a[2:4]

Default bin
---------------
d
```

TRANSITION BINS

Transitional functional point bin is used to examine the legal transitions of a value.
System Verilog allows to specifies one or more sets of ordered value transitions of the coverage point.

Type of Transitions:

- Single Value Transition
- Sequence Of Transitions
- Set Of Transitions
- Consecutive Repetitions
- Range Of Repetition
- Goto Repetition
- Non Consecutive Repetition

Single Value Transition

Single value transition is specified as:
value1 => value2

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y {
bins tran_34 = (3=>4);
bins tran_56 = (5=>6);
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

- In the above example, 2 bins are created for covering the transition of point "y" from 3 to 4 and other for 5 to 6.
- The variable y is given the values and only the transition 5 to 6 is occurring.

```
Coverage report:
-------------------
VARIABLE : cover_point_y
Expected : 2
Covered : 1
Percent: 50.00

Uncovered bins
-----------------
tran_34

Covered bins
---------------
tran_56
```

Sequence Of Transitions

A sequence of transitions is represented as:
value1 => value3 => value4 => value5

In this case, value1 is followed by value3, followed by value4 and followed by value5.
A sequence can be of any arbitrary length.

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y {
bins tran_345 = (3=>4=>5);
bins tran_356 = (3=>5=>6);
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

In the above example, 2 bins re created for covering the transition of point "y" from 3 to 4 to 5 and other for 3 to 5 to 6.
The variable y is given the values and only the transition 3 to 5 to 6 is occurring.

```
Coverage report:
------------------
VARIABLE : cover_point_y
Expected : 2
Covered : 1
Percent: 50.00

Uncovered bins
-----------------
tran_345

Covered bins
---------------
tran_356
```

Set Of Transitions

A set of transitions can be specified as:
range_list1 => range_list2

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{3,5,6};

covergroup cg;
cover_point_y : coverpoint y {
bins trans[] = (3,4=>5,6);
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

In the above example, bin trans creates 4 bin for covering 3=>5,4=>5,3=>6 and 4=>6.

```
Coverage report:
-------------------
VARIABLE : cover_point_y
Expected : 4
Covered : 1
Percent: 25.00

Uncovered bins
-----------------
tran_34_to_56:3->6
tran_34_to_56:4->5
tran_34_to_56:4->6


Covered bins
---------------
tran_34_to_56:3->5
```

Consecutive Repetitions

Consecutive repetitions of transitions are specified using
trans_item [* repeat_range]

Here, trans_item is repeated for repeat_range times. For example,
3 [* 5]
is the same as
3=>3=>3=>3=>3

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{3,3,3,4,4};

covergroup cg;
cover_point_y : coverpoint y {
bins trans_3 = (3[*5]);
bins trans_4 = (4[*2]);
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Coverage report:
------------------
VARIABLE : cover_point_y
Expected : 2
Covered : 1
Percent: 50.00

Uncovered bins
-----------------
trans_3


Covered bins
---------------
trans_4
```

Range Of Repetition

An example of a range of repetition is:
3 [* 3:5]
is the same as
3=>3=>3, 3=>3=>3=>3, 3=>3=>3=>3=>3

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{4,5,3,3,3,3,6,7};

covergroup cg;
cover_point_y : coverpoint y {
bins trans_3[] = (3[*3:5]);
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

In the above example, only the sequence 3=>3=>3=>3 is generated. Other expected sequences 3=>3=>3 and 3=>3=>3=>3=>3 are not generated.

```
Coverage report:
------------------
VARIABLE : cover_point_y
Expected : 3
Covered : 1
Percent: 33.33

Uncovered bins
----------------
tran_3:3[*3]
tran_3:3[*5]

Covered bins
--------------
tran_3:3[*4]
```

Goto Repetition

- The goto repetition is specified using: trans_item [-> repeat_range].
- The required number of occurrences of a particular value is specified by the repeat_range.
- Any number of sample points can occur before the first occurrence of the specified value and any number of sample points can occur between each occurrence of the specified value.
- The transition following the goto repetition must immediately follow the last occurrence of the repetition.

For example:
3 [-> 3]
is the same as
...=>3...=>3...=>3

where the dots (...) represent any transition that does not contain the value 3.

A goto repetition followed by an additional value is represented as follows:
1 => 3 [-> 3] => 5
is the same as
1...=>3...=>3...=>3 =>5

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{1,6,3,6,3,6,3,5};

covergroup cg;
cover_point_y : coverpoint y {
bins trans_3 = (1=>3[->3]=>5);
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Coverage report:
--------------------
VARIABLE : cover_point_y
Expected : 1
Covered : 1
Percent: 100.00
```

Non Consecutive Repetition

- The nonconsecutive repetition is specified using: trans_item [= repeat_range].
- The required number of occurrences of a particular value is specified by the repeat_range.
- Any number of sample points can occur before the first occurrence of the specified value and any number of sample points can occur between each occurrence of the specified value.
- The transition following the nonconsecutive repetition may occur after any number of sample points so long as the repetition value does not occur again.

For example:
3 [= 2]
is same as
...=>3...=>3

A nonconsecutive repetition followed by an additional value is represented as follows:
1 => 3 [=2] => 5
is the same as
1...=>3...=>3...=>5

```
program main;
bit [0:3] y;
bit [0:2] values[$]= '{1,6,3,6,3,6,5};

covergroup cg;
cover_point_y : coverpoint y {
bins trans_3 = (1=>3[=2]=>5);
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Coverage report:
--------------------
VARIABLE : cover_point_y
Expected : 1
Covered : 1
Percent: 100.00
```

WILDCARD BINS

- By default, a value or transition bin definition can specify 4-state values.
- When a bin definition includes an X or Z, it indicates that the bin count should only be incremented when the sampled value has an X or Z in the same bit positions.
- The wildcard bins definition causes all X, Z, or ? to be treated as wildcards for 0 or 1 (similar to the ==? operator).

```
wildcard bins g12_16 = { 4'b11?? };
```

The count of bin g12_16 is incremented when the sampled variable is between 12 and 15:

The count of bin g12_16 is incremented when the sampled variable is between 12 and 15:

1100 1101 1110 1111

```
program main;
reg [0:3] y;
reg [0:3] values[$]= '{ 4'b1100,4'b1101,4'b1110,4'b1111};

covergroup cg;
cover_point_y : coverpoint y {
wildcard bins g12_15 = { 4'b11?? } ;
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Coverage report:
--------------------
VARIABLE : cover_point_y
Expected : 1
Covered : 1
Percent: 100.00

Covered bin
--------------
g12_15
Number of times g12_15 hit : 4
```

Similarly, transition bins can define wildcard bins.

```
wildcard bins T0_3 = (2'b0x => 2'b1x);
```

The count of transition bin T0_3 is incremented for the following transitions (as if by (0,1=>2,3)):

```
00 => 10 , 00 => 11, 01 => 10 , 01 => 11
```

```
program main;
reg [0:1] y;
reg [0:1] values[$]= '{ 2'b00,2'b01,2'b10,2'b11};

covergroup cg;
cover_point_y : coverpoint y {
wildcard bins trans = (2'b0X => 2'b1X );
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Coverage report:
--------------------
VARIABLE : cover_point_y
Expected : 1
Covered : 1
Percent: 100.00

Covered bin
--------------
trans
Number of times trans hit : 1 (01 => 10)
```

IGNORE BINS

A set of values or transitions associated with a coverage-point can be explicitly excluded from coverage by specifying them as ignore_bins.

```
program main;
bit [0:2] y;
bit [0:2] values[$]= '{1,6,3,7,3,4,3,5};

covergroup cg;
cover_point_y : coverpoint y {
ignore_bins ig = {1,2,3,4,5};
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

In the above program, total possible values for y are 0 to 7. Ignore_bins specified to Ignored values between 1 to 5. So the Expected values are 0,6 and 7. Out of these expected values, only 6 and 7 are generated

```
Coverage report:
-------------------
VARIABLE : cover_point_y
Expected : 3
Covered : 2
Percent: 66.66

Uncovered bins
-----------------
auto[0]
```

```
Excluded/Illegal bins
-----------------------
ig
auto[1]
auto[2]
auto[3]
auto[4]
auto[5]

Covered bins
---------------
auto[6]
auto[7]
```

ILLEGAL BINS

- A set of values or transitions associated with a coverage-point can be marked as illegal by specifying them as illegal_bins.
- All values or transitions associated with illegal bins are excluded from coverage. If an illegal value or transition occurs, a runtime error is issued.

```
program main;
bit [0:2] y;
bit [0:2] values[$]= '{1,6,3,7,3,4,3,5};

covergroup cg;
cover_point_y : coverpoint y {
illegal_bins ib = {7};
}

endgroup

cg cg_inst = new();
initial
foreach(values[i])
begin
y = values[i];
cg_inst.sample();
end

endprogram
```

```
Result:
-----------
** ERROR **
Illegal state bin ib of coverpoint cover_point_y in
covergroup cg got hit with value 0x7
```

CROSS COVERAGE

- Cross allows keeping track of information which is received simultaneous on more than one cover point.
- Cross coverage is specified using the cross construct.

```
program main;
bit [0:1] y;
bit [0:1] y_values[$]= '{1,3};

bit [0:1] z;
bit [0:1] z_values[$]= '{1,2};

covergroup cg;
cover_point_y : coverpoint y ;
cover_point_z : coverpoint z ;
cross_yz : cross cover_point_y,cover_point_z ;
endgroup

cg cg_inst = new();
initial
foreach(y_values[i])
begin
y = y_values[i];
z = z_values[i];
cg_inst.sample();
end

endprogram
```

In the above program, y has can have 4 values 0,1,2 and 3 and similarly z can have 4 values 0,1,2 and 3.
The cross product of the y and z will be 16 values (00),(01),(02),(03),(10),(11).....(y,z).....(3,2)(3,3) .
Only combinations (11) and (32) are generated.

```
Cross coverage report: cover points are not shown.

Covered bins
----------------
cover_point_y cover_point_z
auto[3] auto[2]
auto[1] auto[1]
```

User-Defined Cross Bins

User-defined bins for cross coverage are defined using bin select expressions.

```
int i,j;
covergroup ct;
coverpoint i { bins i[] = { [0:1] }; }
coverpoint j { bins j[] = { [0:1] }; }
x1: cross i,j;
x2: cross i,j {
bins i_zero = binsof(i) intersect { 0 };
}
endgroup
```

```
Cross x1 has the following bins:
<i[0],j[0]>
<i[1],j[0]>
<i[0],j[1]>
<i[1],j[1]>
Cross x2 has the following bins:
i_zero
<i[1],j[0]>
<i[1],j[1]>
```